

AssanPay APIs — Signed Request Manual (Indonesia)

Full-security integration with HMAC-SHA256 request signatures. Every request is authenticated with your API key plus a cryptographic signature derived from your API secret.

API Index

POST	/api/checkout/sessions	Checkout Session API
POST	/api/checkout/sessions	Checkout Session Encrypted API
POST	/api/payments	H2H Pay-in API
POST	/api/payments	H2H Pay-in Encrypted API
GET	/api/merchant/status-inquiry?type=payin&orderId={{orderId}}	Pay-in Status Inquiry
POST	/api/merchant/wallet-payouts	H2H Payout API
POST	/api/merchant/wallet-payouts	H2H Payout Encrypted API
GET	/api/merchant/status-inquiry?type=payout&orderId={{orderId}}	Payout Status Inquiry
GET	/api/merchant/balance/h2h	Balance Check API
WEBHOOK	Callback Signature Verification	Callback Signature Verification
WEBHOOK	Pay-in Callback	Pay-in Callback
WEBHOOK	Payout Callback	Payout Callback

Overview

This manual guides you through implementing AssanPay's Host-to-Host (H2H) APIs with IP allowlist + API Secret authentication. Every sensitive endpoint requires HMAC-SHA256 request signatures in addition to the API key and IP check.

- branchCode is required in every request. Create branches from the Branches page, and each branch is approved by admin.
- Your server IPs must be whitelisted before H2H APIs can be used.
- API key and API secret are generated from the Settings page.
- Status inquiry endpoints are exempt from HMAC signing. A valid X-API-KEY alone is sufficient.
- Final response status codes are listed in the Status Codes section at the end of this manual.
- For callback endpoints, return any 2xx response to acknowledge receipt and avoid retries.
- Request encryption can use the sub-merchant API secret, but response decryption and callback signature generation must use the main merchant API secret.

Auth Modes

AUTH MODE	REQUIRES HMAC SIGNATURE	REQUIRES IP ALLOWLIST
HMAC_ONLY	Yes	No
IP_ONLY	No	Yes
HMAC_PLUS_IP	Yes	Yes

Request Signature

Cryptographic verification of every request

A signature proves that the holder of the API secret produced this exact request, and that the request has not been tampered with or replayed. Build a canonical string from five parts joined by newline characters, then HMAC it with your secret.

- **TIMESTAMP** is Unix time in seconds (10 digits). The server rejects any request more than ± 300 seconds from server time, so keep your clock in sync via NTP.
- **NONCE** is a unique random string per request (a UUID is ideal). Each nonce may be used once per merchant; a repeat is rejected with `INVALID_NONCE`.
- **PATH_WITH_QUERY** is the request path including the query string, with no scheme or host — e.g. `/api/payments` or `/api/payments?foo=bar`.
- For an empty body, hash the empty string.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Signature Formula

Signature Formula

```
bodyHash = SHA256_HEX(rawBody)
canonical = METHOD + "\n" + PATH_WITH_QUERY + "\n" + TIMESTAMP + "\n" + NONCE + "\n"
+ BODY_HASH
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

Worked Example

INPUT	SAMPLE VALUE
Method	POST
Path with Query	/api/payments
Timestamp	1720512000
Nonce	b3f1c2a4-9d7e-4c11-8a2b-1e5f6a7c8d90
Request Body	{ "- branchCode": "BR001", "paymentMethodName": "QRIS", "amount": 50000, "orderId": "ORD-100001"
API Secret	sk_test_secret_example_key
Body Hash (SHA-256 hex)	85624b12e0346df106b7a611c22925de8ec0cb3941eccfe9836af5343189db8d
X-SIGNATURE (expected)	G09q9i14fXQxG0XDPIbEU2+DFjmf95dv9IP/faYr lE=

If you reproduce that exact signature with your own code, your signer is correct.

Request Headers

Request Headers

```
X-API-KEY: your_api_key
X-TIMESTAMP: 1720512000
X-NONCE: b3f1c2a4-9d7e-4c11-8a2b-1e5f6a7c8d90
X-SIGNATURE: G09q9i14fXQxG0XDPIbEU2+DFjmfr95dv9IP/faYrLE=
Content-Type: application/json
```

Implementation Example

Implementation Example

```
function generateRequestSignature(
  { method, path, timestamp, nonce, rawBody },
  apiSecret
) {
  const bodyString = rawBody || '';

  // Step 1: Body hash (SHA256 hex)
  const bodyHash = crypto.createHash('sha256')
    .update(bodyString)
    .digest('hex');

  // Step 2: Canonical string
  const canonical = `${method}\n${path}\n${timestamp}\n${nonce}\n${bodyHash}`;

  // Step 3: Signature (HMAC SHA256 -> Base64)
  const signature = crypto.createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { signature, canonical, bodyHash };
}
```

Sending a Signed Request

● ● ● Sending a Signed Request

```
const API_KEY    = 'your_api_key';
const API_SECRET = 'your_api_secret';
const BASE_URL   = 'https://api.example-ph.com';

async function signedPost(path, bodyObj) {
  // Serialize ONCE; sign and send the SAME string.
  const rawBody   = JSON.stringify(bodyObj);
  const timestamp = String(Math.floor(Date.now() / 1000));
  const nonce     = crypto.randomUUID();

  const bodyHash = crypto.createHash('sha256')
    .update(rawBody, 'utf8')
    .digest('hex');

  const canonical = ['POST', path, timestamp, nonce, bodyHash].join('\n');
  const signature = crypto.createHmac('sha256', API_SECRET)
    .update(canonical, 'utf8')
    .digest('base64');

  return fetch(BASE_URL + path, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-API-KEY':    API_KEY,
      'X-SIGNATURE':  signature,
      'X-TIMESTAMP':  timestamp,
      'X-NONCE':      nonce
    },
    body: rawBody // exact signed bytes
  });
}

signedPost('/api/payments', {
  branchCode: 'BR001',
  paymentMethodName: 'QRIS',
  amount: 50000,
  orderId: 'ORD-100001',
  customerContact: '081234567890'
});
```

Checkout Session API

Hosted checkout page for customer-initiated pay-ins

POST /api/checkout/sessions

Generates a unique checkout session URL per transaction. Redirect the customer to checkoutUrl to complete payment on the hosted page.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Amount to collect in IDR. Whole rupiah only — fractional amounts are rejected, not rounded. Minimum 10,000.
branchCode	string	Required	Merchant branch code. Must match your API key if the key is branch-scoped.
customerEmail	string	Required	Customer email. Must be non-blank.
customerContact	string	Required	Customer phone/contact. Must be non-blank.
paymentMethodName	string	Required	One of: QRIS, OVO, DANA, GOPAY, LINKAJA, SHOPEEPAY, VA. Must be enabled for your merchant.
bankCode	string	Optional	VA only, and REQUIRED when paymentMethodName is VA. The collection bank: BCA, BRI, BNI, MANDIRI or CIMB, and it must be enabled for your merchant. Ignored for every other method.
orderId	string	Optional	Strongly recommended. Max 32 characters, letters and digits only (no special characters). At most one session per order ID.
customerName	string	Optional	Customer display name.
callbackUrl	string	Optional	Strongly recommended. Where we POST the final callback.
successUrl	string	Optional	Browser redirect target on success.
failedUrl	string	Optional	Browser redirect target on failure.
pendingUrl	string	Optional	Browser redirect target while pending.
expiresInSeconds	number	Optional	Session lifetime from creation. Defaults to 600 (10 min) and is capped at the server maximum.

Request Body

Request Body

```
{
  "orderId": "ORD-100001",
  "amount": 50000,
  "currency": "IDR",
  "branchCode": "BR001",
  "customerName": "Budi Santoso",
  "customerEmail": "budi@example.com",
  "customerContact": "081234567890",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay",
  "successUrl": "https://merchant.example.com/checkout/success",
  "failedUrl": "https://merchant.example.com/checkout/failed",
  "pendingUrl": "https://merchant.example.com/checkout/pending",
  "expiresInSeconds": 600,
  "paymentMethodName": "QRIS"
}
```

Success Response (201 Created)

Success Response (201 Created)

```
{
  "sessionId": "5f9c2b7e-1a34-4c9d-8e21-0b7a6f4d3c22",
  "orderId": "ORD-100001",
  "paymentId": 90210,
  "checkoutUrl":
    "https://business.assanpay.com/pay?sessionId=5f9c2b7e-1a34-4c9d-8e21-0b7a6f4d3c22",
  "expiresAt": 1720512600
}
```

Failure Responses

INVALID_BRANCH_CODE

```
{
  "success": false,
  "errorCode": "INVALID_BRANCH_CODE",
  "message": "Branch code is required"
}
```

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Order id must be at most 32 chars (letters and digits only)"
}
```

INVALID_SIGNATURE

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

MIN_AMOUNT_NOT_MET

```
{
  "success": false,
  "errorCode": "MIN_AMOUNT_NOT_MET",
  "message": "Minimum amount not met"
}
```

Checkout Session Encrypted API

AES-256-GCM encrypted checkout payload — signature required

POST /api/checkout/sessions

Signature on encrypted body

The signature must be generated using the final encrypted request body, not the plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "orderId": "ORD-100001",
  "amount": 50000,
  "currency": "IDR",
  "branchCode": "BR001",
  "customerName": "Budi Santoso",
  "customerEmail": "budi@example.com",
  "customerContact": "081234567890",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay",
  "successUrl": "https://merchant.example.com/checkout/success",
  "failedUrl": "https://merchant.example.com/checkout/failed",
  "pendingUrl": "https://merchant.example.com/checkout/pending",
  "expiresInSeconds": 600,
  "paymentMethodName": "QRIS"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response

Success Response

```
{
  "sessionId": "5f9c2b7e-1a34-4c9d-8e21-0b7a6f4d3c22",
  "orderId": "ORD-100001",
  "paymentId": 90210,
  "checkoutUrl":
    "https://business.assanpay.com/pay?sessionId=5f9c2b7e-1a34-4c9d-8e21-0b7a6f4d3c22",
  "expiresAt": 1720512600
}
```

H2H Pay-in API

Server-to-server pay-in — final status arrives via callback

POST /api/payments

Your backend creates the payment directly; there is no hosted page of ours. The response carries whatever the chosen rail needs: for QRIS an EMVCo payload in qrPayload, for every other method a provider-hosted link in redirectUrl.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
branchCode	string	Required	Merchant branch code. Must match your API key if the key is branch-scoped.
paymentMethodName	string	Required	One of: QRIS, OVO, DANA, GOPAY, LINKAJA, SHOPEEPAY, VA. Must be enabled for your merchant.
amount	number	Required	Amount in IDR. Whole rupiah only — fractional amounts are rejected, not rounded. Minimum 10,000.
bankCode	string	Optional	VA only, and REQUIRED when paymentMethodName is VA. The collection bank: BCA, BRI, BNI, MANDIRI or CIMB, and it must be enabled for your merchant. Ignored for every other method.
customerContact	string	Required	Customer mobile number, e.g. 081234567890 (local, 08 prefix) or 6281234567890 (international, 62 prefix).
orderId	string	Optional	Strongly recommended. Max 32 characters, letters and digits only (no special characters). Duplicate returns DUPLICATE_ORDER.
customerEmail	string	Optional	Customer email. A placeholder is stored if blank.
customerName	string	Optional	Customer name. A synthetic name is stored if blank.
callbackUrl	string	Optional	Strongly recommended. Where we POST the final SUCCESS/FAILED callback.

Request Body

Request Body

```
{
  "branchCode": "BR001",
  "paymentMethodName": "QRIS",
  "amount": 50000,
  "currency": "IDR",
  "orderId": "ORD-100001",
  "customerContact": "081234567890",
  "customerName": "Budi Santoso",
  "customerEmail": "budi@example.com",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay"
}
```

Success Response — QRIS (201 Created)

Success Response — QRIS (201 Created)

```
{
  "success": true,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Awaiting payment",
  "type": "payin",
  "orderId": "ORD-100001",
  "paymentId": "T2026070912000042",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200042",
  "rail": "QR",
  "qrPayload": "00020101021226670016COM.NOBUBANK.WWW0118936005031260000200214...",
  "redirectUrl": null,
  "qrImage": null,
  "occurredAt": "2026-07-09T12:00:00+07:00"
}
```

Success Response — e-wallet (201 Created)

Success Response — e-wallet (201 Created)

```
{
  "success": true,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Awaiting payment",
  "type": "payin",
  "orderId": "ORD-100002",
  "paymentId": "T2026070912000043",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200043",
  "rail": "EWALLET",
  "qrPayload": null,
  "redirectUrl": "https://pay.example-psp.com/h5/checkout/9f3c1a...",
  "qrImage": null,
  "occurredAt": "2026-07-09T12:00:00+07:00"
}
```

Success Response — virtual account (201 Created)

```
Success Response – virtual account (201 Created)

{
  "success": true,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Awaiting payment",
  "type": "payin",
  "orderId": "ORD-100003",
  "paymentId": "T2026070912000044",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200044",
  "rail": "VA",
  "qrPayload": null,
  "redirectUrl": "https://pay.example-psp.com/h5/checkout/1b7d4e...",
  "vaNumber": "8881000012345678",
  "vaBankCode": "BCA",
  "vaExpiry": "2026-07-09T13:00:00+07:00",
  "qrImage": null,
  "occurredAt": "2026-07-09T12:00:00+07:00"
}
```

What to show the payer: qrPayload for QRIS, redirectUrl for everything else

Exactly one of these two carries the payment instrument, and which one depends on the rail. QRIS returns qrPayload — the raw EMVCo string — with redirectUrl null: render it as a QR code yourself (any QR library) and show it to the payer. Every other method returns redirectUrl with qrPayload null: send the payer to that URL, where the provider hosts the payment page. VA additionally returns vaNumber, vaBankCode and vaExpiry, so you may display the account number instead of redirecting. Take whichever you get from the create response and store it before responding to your own caller.

A PENDING response is not final

A PENDING create response only means 'accepted'. The authoritative outcome is the callback, or a status inquiry. We never send a callback for a PENDING state.

Failure Responses

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers"
}
```

INVALID_SIGNATURE

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

REQUEST_EXPIRED

```
{
  "success": false,
  "errorCode": "REQUEST_EXPIRED",
  "message": "Request expired"
}
```

INVALID_NONCE

```
{
  "success": false,
  "errorCode": "INVALID_NONCE",
  "message": "Nonce already used"
}
```

VALIDATION_ERROR

```
{
  "success": false,
  "errorCode": "VALIDATION_ERROR",
  "message": "amount: must be greater than or equal to 0.01; branchCode: must not be blank"
}
```

● ● ● DUPLICATE_ORDER (409)

```
{
  "success": false,
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists"
}
```

H2H Pay-in Encrypted API

AES-256-GCM encrypted payload transmission

POST /api/payments

Signature on encrypted body

The signature must be generated using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "branchCode": "BR001",
  "paymentMethodName": "QRIS",
  "amount": 50000,
  "currency": "IDR",
  "orderId": "ORD-100001",
  "customerContact": "081234567890",
  "customerName": "Budi Santoso",
  "customerEmail": "budi@example.com",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response (Encrypted)

Success Response (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Failure Responses

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers",
  "details": null,
  "metadata": null,
  "timestamp": "2026-07-09T12:00:00"
}
```

ENCRIPTION_NOT_ENABLED

```
{
  "success": false,
  "errorCode": "ENCRIPTION_NOT_ENABLED",
  "message": "Encryption not enabled"
}
```

CRYPTO_ERROR

```
{
  "success": false,
  "errorCode": "CRYPTO_ERROR",
  "message": "Decryption failed"
}
```

Pay-in Status Inquiry

Query the current state of a pay-in by order ID

GET /api/merchant/status-inquiry?type=payin&orderId={{orderId}}

Exempt from HMAC

Status inquiry endpoints require only a valid X-API-KEY. No signature, timestamp, or nonce is needed, even when your auth mode includes HMAC.

Use this when you missed a callback or need to reconcile. There is no request body — pass the order ID as a query parameter.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

Success Response

● ● ● Success Response

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payment completed",
  "type": "payin",
  "orderId": "ORD-100001",
  "paymentId": "T2026070912000042",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200042",
  "reference": "SP-9931200042",
  "rail": "QR",
  "reason": null,
  "occurredAt": "2026-07-09T12:00:00+07:00",
  "successAt": "2026-07-09T12:03:10+07:00"
}
```

Failed Transaction Response

● ● ● Failed Transaction Response

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Insufficient funds",
  "type": "payin",
  "orderId": "ORD-100002",
  "paymentId": "T2026070912050099",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200099",
  "reference": null,
  "rail": "QR",
  "reason": "Insufficient funds",
  "occurredAt": "2026-07-09T12:05:00+07:00",
  "failedAt": "2026-07-09T12:05:40+07:00"
}
```

Not Found Response

Not Found Response

```
{
  "success": false,
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Payment not found",
  "timestamp": "2026-07-09T12:00:00"
}
```

H2H Payout API

Disburse funds from your settlement balance to a beneficiary bank account

POST /api/merchant/wallet-payouts

Rail selection is automatic

You do not choose the rail. It is derived from the destination you send: an e-wallet code (OVO, DANA, GOPAY, LINKAJA, SHOPEEPAY) settles as WALLET, anything else as BANK. Amount does not affect rail selection.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
amount	number	Required	Payout amount in IDR. Whole rupiah only — fractional amounts are rejected, not rounded. Minimum 10,000.
orderId	string	Required	Your reference. Max 32 characters, letters and digits only (no special characters). Used for status inquiry and de-duplication.
branchCode	string	Required	Merchant branch code. Required and enforced if your API key is branch-scoped.
accountNo	string	Required	Beneficiary account number.
bankName	string	Required	Beneficiary bank name.
bankCode	string	Required	Beneficiary destination code from the Payout Bank Codes directory (e.g. 1007 for Bank BCA, 1902 for DANA). Legacy raw provider codes are accepted as aliases; bank names are never accepted as codes.
beneficiaryName	string	Required	Account holder name.
purpose	string	Optional	Payout purpose or description.
callbackUrl	string	Optional	Strongly recommended. Where we POST the final SUCCESS/FAILED callback.

Request Body

Request Body

```
{
  "amount": 75000,
  "orderId": "PO-200045",
  "branchCode": "BR001",
  "bankName": "Bank Central Asia",
  "bankCode": "1007",
  "accountNo": "1234567890",
  "beneficiaryName": "Siti Rahayu",
  "purpose": "Supplier settlement",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay"
}
```

Success Response — BANK (bank account destination)

● ● ● Success Response – BANK (bank account destination)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payout completed",
  "type": "payout",
  "orderId": "PO-200045",
  "payoutId": 44021,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": "SP-PO-778812",
  "reference": "SP-PO-778812",
  "accountNumber": "1234567890",
  "accountTitle": "Siti Rahayu",
  "rail": "BANK",
  "occurredAt": "2026-07-09T12:00:00+07:00",
  "successAt": "2026-07-09T12:04:55+07:00"
}
```

Success Response — WALLET (e-wallet destination)

●●● Success Response – WALLET (e-wallet destination)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payout completed",
  "type": "payout",
  "orderId": "PO-200046",
  "payoutId": 44022,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": "KP-PO-778890",
  "reference": "KP-PO-778890",
  "accountNumber": "1234567890",
  "accountTitle": "Siti Rahayu",
  "rail": "WALLET",
  "occurredAt": "2026-07-09T12:00:00+07:00",
  "successAt": "2026-07-09T14:10:02+07:00"
}
```

Pending Response

● ● ● Pending Response

```
{
  "success": false,
  "status": "PENDING",
  "statusCode": "202",
  "message": "PSP processing",
  "type": "payout",
  "orderId": "PO-200045",
  "payoutId": 44021,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": "SP-PO-778812",
  "reference": "SP-PO-778812",
  "accountNumber": "1234567890",
  "accountTitle": "Siti Rahayu",
  "rail": "BANK",
  "occurredAt": "2026-07-09T12:00:00+07:00"
}
```

Failed Response

Failed Response

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Beneficiary account invalid",
  "type": "payout",
  "orderId": "PO-200047",
  "payoutId": 44023,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": null,
  "reference": null,
  "accountNumber": "1234567899",
  "accountTitle": null,
  "rail": "BANK",
  "reason": "Beneficiary account invalid",
  "occurredAt": "2026-07-09T12:10:00+07:00",
  "failedAt": "2026-07-09T12:10:31+07:00"
}
```

A PENDING response is not final

A PENDING create response only means 'accepted'. The authoritative outcome is the callback, or a status inquiry. We never send a callback for a PENDING state.

Payout amount limits come from your merchant configuration: `payoutMinAmount`, `payoutMaxAmount`, `payoutDailyLimit`, and `payoutMonthlyLimit`.

Failure Responses

MIN_AMOUNT_NOT_MET

```
{
  "success": false,
  "errorCode": "MIN_AMOUNT_NOT_MET",
  "message": "Minimum amount not met"
}
```

● ● ● PER_TRANSACTION_LIMIT_EXCEEDED

```
{
  "success": false,
  "errorCode": "PER_TRANSACTION_LIMIT_EXCEEDED",
  "message": "Per transaction limit exceeded"
}
```

● ● ● MONTHLY_LIMIT_EXCEEDED

```
{
  "success": false,
  "errorCode": "MONTHLY_LIMIT_EXCEEDED",
  "message": "Monthly payout limit exceeded"
}
```

● ● ● INVALID_REQUEST (unknown bank code)

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Unknown payout bank code"
}
```

● ● ● OPERATION_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "OPERATION_NOT_ALLOWED",
  "message": "Account number must match merchant payout account"
}
```

● ● ● INVALID_BRANCH_CODE

```
{
  "success": false,
  "errorCode": "INVALID_BRANCH_CODE",
  "message": "branchCode does not match authenticated sub merchant"
}
```

H2H Payout Encrypted API

Encrypted payout with AES-256-GCM

POST /api/merchant/wallet-payouts

Signature on encrypted body

Generate the signature using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "amount": 75000,
  "orderId": "PO-200045",
  "branchCode": "BR001",
  "bankName": "Bank Central Asia",
  "bankCode": "1007",
  "accountNo": "1234567890",
  "beneficiaryName": "Siti Rahayu",
  "purpose": "Supplier settlement",
  "callbackUrl": "https://merchant.example.com/webhooks/assanpay"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Payout Status Inquiry

Query the current state of a payout by order ID

GET /api/merchant/status-inquiry?type=payout&orderId={{orderId}}

Exempt from HMAC

A valid X-API-KEY alone is sufficient. A payout that reached COMPLETED is reported here as status SUCCESS.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

No request body required for this endpoint.

Success Response

Success Response

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payout completed",
  "type": "payout",
  "orderId": "PO-200045",
  "payoutId": 44021,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": "SP-PO-778812",
  "reference": "SP-PO-778812",
  "accountNumber": "1234567890",
  "accountTitle": "Siti Rahayu",
  "rail": "BANK",
  "successAt": "2026-07-09T12:04:55+07:00"
}
```

Failure Responses

RESOURCE_NOT_FOUND

```
{
  "success": false,
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Payout not found"
}
```

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Order id is required"
}
```

Balance Check API

Check merchant settlement balance from your system

GET /api/merchant/balance/h2h

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Rejected if more than ± 300 s from server time.
X-NONCE	Required	Unique random value per request (a UUID). Never reuse a nonce.
X-SIGNATURE	Required	Generated request signature

Success Response

Success Response

```
{
  "availableBalance": 148250.75,
  "balanceAmount": 148250.75,
  "branchesByCode": {
    "BR001": {
      "amount": 148250,
      "branchName": "Manila Branch",
      "ledgerBalance": 148250.75,
      "unsettledSuccessfulAmount": 0,
      "unsettledSuccessfulCount": 0
    },
    "BR002": {
      "amount": 0,
      "branchName": "Cebu Branch",
      "ledgerBalance": 0.00,
      "unsettledSuccessfulAmount": 0,
      "unsettledSuccessfulCount": 0
    }
  },
  "currency": "IDR",
  "payoutFreeze": false,
  "pendingBalance": 0,
  "unsettledSuccessfulAmount": 0,
  "unsettledSuccessfulCount": 0,
  "updatedAt": "2026-07-09T12:00:23.598585+07:00"
}
```

Callback Signature Verification

Callbacks use a different scheme from your outbound requests

A callback signature verifies that the webhook was sent by an authorized source and that its content has not been tampered with. When a webhook is received, generate a signature from the received values and compare it against the signature sent in the request headers.

This is NOT the request signing scheme

The callback canonical string has only three parts: EVENT_ID, TIMESTAMP, and RAW_BODY. It uses the event ID (not METHOD, PATH, or NONCE) and it embeds the raw body directly — it does not SHA-256 the body first. The header names are the X-Assanpay-* family, not X-SIGNATURE / X-TIMESTAMP / X-NONCE. This asymmetry trips up almost everyone.

- Event ID: sent in header X-Assanpay-Event-Id (unique identifier for the event).
- Timestamp: sent in header X-Assanpay-Timestamp (Unix timestamp in seconds).
- Raw request body: the exact raw body string received in the webhook (do not parse or modify before hashing).
- API Secret: your shared secret key used to verify the signature.

Important: do not modify the body

Use the exact raw request body as received. Do not parse JSON, re-stringify, pretty-print, or change whitespace/newlines before generating the signature.

Encryption asymmetry — verify BEFORE you decrypt

If your account has response encryption enabled, the body we POST is an encrypted envelope, and the signature is computed over the exact bytes we transmit. Verify the signature against the raw received body first, and only then decrypt. If you decrypt first and verify the plaintext, verification will fail.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Verification Flow

1

Extract required headers

Read X-Assanpay-Event-Id, X-Assanpay-Timestamp, and X-Assanpay-Signature from the incoming webhook headers.

2

Build the canonical string

Combine EVENT_ID, TIMESTAMP, and RAW_BODY in this exact order, separated by a newline (\n).

3

Generate HMAC

Compute Base64(HMAC-SHA256(canonical, API_SECRET)).

4

Compare signatures

Use a constant-time comparison. If the generated signature matches the received X-Assanpay-Signature, treat the webhook as valid.

Signature Formula

Signature Formula

```
canonical = EVENT_ID + "\n" + TIMESTAMP + "\n" + RAW_BODY
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

Worked Example

INPUT	SAMPLE VALUE
Event ID	8f2a1c9e-4b6d-47a2-9c3e-0d1f2a3b4c5d
Timestamp	1720512130
Raw Body	{"- success":true,"status":"SUCCESS","statusCode":"200","type":"payin","orderId":"ORD-1
API Secret	sk_test_secret_example_key
X-Assanpay-Signature (expected)	0vmbboLR9YhVvul85cz5YDGm2sbfHwN0DdxjN7vpdn0=

Verification Example

Verification Example

```
function generateCallbackSignature(rawBody, headers, apiSecret) {
  if (!apiSecret) throw new Error('Missing API secret');

  const eventId = headers['X-Assanpay-Event-Id'] || '';
  const timestamp = headers['X-Assanpay-Timestamp'] || '';
  const body = rawBody ? String(rawBody) : '';

  const canonical = `${eventId}\n${timestamp}\n${body}`;

  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { canonical, signature };
}

function verifyCallbackSignature(rawBody, headers, apiSecret) {
  const generated = generateCallbackSignature(rawBody, headers, apiSecret);
  const receivedSignature = headers['X-Assanpay-Signature'] || '';
  return generated.signature === receivedSignature;
}
```

Pay-in Callback

Webhook for final pay-in status. Return any 2xx response to acknowledge receipt.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried 3 times: first after 15 seconds, then after 30 seconds, then after 60 seconds. 408, 429, 5xx, timeouts and connection errors are retried; other 4xx responses are treated as non-retryable.

Success and Failed share one shape

Both callbacks send the same JSON body. Tell them apart by status (SUCCESS vs FAILED) or statusCode (200 vs 400). On success reason is null; on failure reason is non-null.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id. Use it as your de-duplication key.
X-Assanpay-Timestamp	Required	Unix timestamp in seconds at which we sent the callback
X-Assanpay-Signature	Required	Base64(HMAC-SHA256(eventId + \n + timestamp + \n + rawBody, API_SECRET))
X-Assanpay-Delivery-Seq	Optional	Monotonic delivery sequence for replays. Present when greater than 0.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON)

```
Callback Body (Success JSON)

{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payment completed",
  "type": "payin",
  "orderId": "ORD-100001",
  "paymentId": "T2026070912000042",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200042",
  "reference": "SP-9931200042",
  "accountNumber": null,
  "accountTitle": null,
  "rail": "QR",
  "qrPayload": null,
  "redirectUrl": null,
  "reason": null,
  "occurredAt": "2026-07-09T12:00:00+07:00",
  "successAt": "2026-07-09T12:03:10+07:00"
}
```

Callback Body (Failed JSON)

Callback Body (Failed JSON)

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Insufficient funds",
  "type": "payin",
  "orderId": "ORD-100002",
  "paymentId": "T2026070912050099",
  "amount": 50000,
  "currency": "IDR",
  "transactionId": "SP-9931200099",
  "reference": null,
  "accountNumber": null,
  "accountTitle": null,
  "rail": "QR",
  "qrPayload": null,
  "redirectUrl": null,
  "reason": "Insufficient funds",
  "occurredAt": "2026-07-09T12:05:00+07:00",
  "failedAt": "2026-07-09T12:05:40+07:00"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "j6mKSr9XrvmoLKD5",
  "cipherText": "BCRgnjSPoPuHF1w4tT9DJ58....."
}
```

Payout Callback

Webhook for final payout status. Return any 2xx response to acknowledge receipt.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried 3 times: first after 15 seconds, then after 30 seconds, then after 60 seconds. 408, 429, 5xx, timeouts and connection errors are retried; other 4xx responses are treated as non-retryable.

COMPLETED is reported as SUCCESS

A payout that reached COMPLETED is delivered with status SUCCESS and statusCode 200. Only SUCCESS and FAILED are webhooked.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id. Use it as your de-duplication key.
X-Assanpay-Timestamp	Required	Unix timestamp in seconds at which we sent the callback
X-Assanpay-Signature	Required	Base64(HMAC-SHA256(eventId + \n + timestamp + \n + rawBody, API_SECRET))
X-Assanpay-Delivery-Seq	Optional	Monotonic delivery sequence for replays. Present when greater than 0.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON)

Callback Body (Success JSON)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payout completed",
  "type": "payout",
  "orderId": "PO-200045",
  "payoutId": 44021,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": "SP-PO-778812",
  "reference": "SP-PO-778812",
  "accountNumber": "1234567890",
  "accountTitle": "Siti Rahayu",
  "rail": "BANK",
  "reason": null,
  "occurredAt": "2026-07-09T12:00:00+07:00",
  "successAt": "2026-07-09T12:04:55+07:00"
}
```

Callback Body (Failed JSON)

Callback Body (Failed JSON)

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Beneficiary account invalid",
  "type": "payout",
  "orderId": "PO-200047",
  "payoutId": 44023,
  "amount": 75000,
  "currency": "IDR",
  "transactionId": null,
  "reference": null,
  "accountNumber": "1234567899",
  "accountTitle": null,
  "rail": "BANK",
  "reason": "Beneficiary account invalid",
  "occurredAt": "2026-07-09T12:10:00+07:00",
  "failedAt": "2026-07-09T12:10:31+07:00"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "j6mKSr9XrvmoLKD5",
  "cipherText": "BCRgnjSPoPuHF1w4tT9DJ58....."
}
```

Error Response Shape & Codes

Every error from every endpoint uses one of two closely related shapes

Most failures — bad input, limit breaches, not-found, duplicates — come back as a business error. Branch on `errorCode`, never on the message text. Fields that are null may be omitted from the JSON entirely.

Business / Validation Error

Business / Validation Error

```
{
  "success": false,
  "errorCode": "MIN_AMOUNT_NOT_MET",
  "message": "Minimum amount not met",
  "details": null,
  "metadata": null,
  "timestamp": "2026-07-09T12:00:00"
}
```

Bean-validation Error (H2H Pay-in only)

Bean-validation Error (H2H Pay-in only)

```
{
  "success": false,
  "errorCode": "VALIDATION_ERROR",
  "message": "amount: must be greater than or equal to 0.01; branchCode: must not be blank",
  "details": null,
  "timestamp": "2026-07-09T12:00:00"
}
```

Auth Error (written by the security filter)

Auth Error (written by the security filter)

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

Error Code Reference

ERROR CODE	TYPICAL HTTP	MEANING
AUTH_REQUIRED	401	Missing API key, or no API secret on file.
INVALID_API_KEY	401	API key unknown or merchant blocked.

MISSING_AUTH_HEADERS	401	Missing X-SIGNATURE or X-TIMESTAMP.
MISSING_NONCE	401	Missing X-NONCE.
INVALID_TIMESTAMP	401	X-TIMESTAMP is not a number.
REQUEST_EXPIRED	401	Timestamp outside the ± 300 s window.
INVALID_SIGNATURE	401	HMAC mismatch.
INVALID_NONCE	401	Nonce already used for this merchant.
IP_NOT_ALLOWED	401	Caller IP not in the allowlist.
SUB_MERCHANT_BLOCKED	401	Sub-merchant blocked or unapproved.
ENCRYPTED_PAYLOAD_REQUIRED	400	Encryption enabled but body was plaintext.
ENCRYPTION_NOT_ENABLED	400	Encrypted body sent but encryption is not enabled.
CRYPTO_ERROR	400	Decryption failed.
INVALID_REQUEST	400	Generic bad input: bad order ID, missing field, rail conflict.
VALIDATION_ERROR	400	Bean-validation failure. Only H2H pay-in returns this.
INVALID_BRANCH_CODE	400	branchCode missing, unknown, or mismatched.
PAYMENT_METHOD_NOT_ALLOWED	400	No enabled method can accept this request.
MERCHANT_BLOCKED	400	Your merchant account is blocked.
MIN_AMOUNT_NOT_MET	400	Amount below the configured minimum.
PER_TRANSACTION_LIMIT_EXCEEDED	400	Amount above the per-transaction maximum.
DAILY_LIMIT_EXHAUSTED	400	This transaction would exceed today's cap.
MONTHLY_LIMIT_EXCEEDED	400	This transaction would exceed the month's cap.
DUPLICATE_ORDER	409	Order ID already exists (pay-in).

RESOURCE_NOT_FOUND	404	Order, payment, or payout not found.
OPERATION_NOT_ALLOWED	403	Action not permitted, e.g. payout account mismatch.
ACCESS_DENIED	403	API key lacks the authority for this endpoint.
INTERNAL_ERROR	500	Unexpected server error.

Status Codes & Lifecycle

Reference for response status codes and transaction states

	200 Status Code	Success
	202 Status Code	Pending
	400 Status Code	Failed
	404 Status Code	Unknown

statusCode mapping: SUCCESS and COMPLETED map to 200; PENDING and INITIATED map to 202; FAILED maps to 400; unknown or absent maps to 404.

Pay-in (payment) statuses

STATUS	FINAL?	WEBHOOKED?	MEANING
INITIATED	No	No	Created, not yet processing.
PENDING	No	No	Awaiting customer or provider action.
SUCCESS	Yes	Yes	Paid.
FAILED	Yes	Yes	Did not complete.

Payout statuses

STATUS	FINAL?	REPORTED AS	MEANING
PENDING	No	PENDING	Accepted, or at the provider.
EXPIRED	Terminal-ish	EXPIRED	Timed out without completion.
COMPLETED	Yes	SUCCESS	Funds disbursed.
FAILED	Yes	FAILED	Disbursement failed.

Only SUCCESS (a payout that reached COMPLETED) and FAILED are delivered as callbacks. For anything still PENDING or EXPIRED, poll the status inquiry endpoint.

Troubleshooting

The integration bugs we see most often

INVALID_SIGNATURE on every request

The number-one cause is signing bytes that differ from the bytes you send. Serialize your JSON once and use that exact string for both the SHA-256 hash and the HTTP body. Confirm you hashed the body rather than embedding it, joined the five parts with real newline characters in the right order, and Base64-encoded the HMAC rather than hex.

My webhook signature check fails but my request signing works

You are almost certainly applying the request scheme to the callback. Callbacks use eventId + newline + timestamp + newline + rawBody — the raw body is embedded, not hashed — and the X-Assanpay-* headers. Verify against the raw received bytes, and if response encryption is enabled, verify before decrypting.

- **REQUEST_EXPIRED** — your X-TIMESTAMP is more than 5 minutes from server time. Sync your clock via NTP and send Unix seconds.
- **INVALID_NONCE** — you reused a nonce. Generate a fresh UUID for every request.
- No callback for a pending payment — by design. Only **SUCCESS** and **FAILED** are webhooked. Poll the status inquiry for in-flight state.
- **ENCRYPTED_PAYLOAD_REQUIRED / ENCRYPTION_NOT_ENABLED** — your account's request-encryption setting doesn't match what you sent. Coordinate the setting with us during onboarding.
- **ACCESS_DENIED** — your API key lacks authority for that endpoint, or you're calling a portal-only route. Use the documented merchant endpoints with the API key issued for API access.
- Status inquiry says 'Payment not found' but you just created it — check you're querying the exact, trimmed orderId under the same merchant/branch scope as the key that created it. Branch-scoped keys only see their own branch's transactions.
- Which payout rail will be used — it is derived from the destination: an e-wallet code settles as **WALLET**, anything else as **BANK**. Do not send provider, rail, or iban.

Payout Bank Codes

Destination codes for the bankCode field on a payout

Send the destination as bankCode when creating a wallet payout. You never pick the rail: it is derived from the destination — an e-wallet settles as **WALLET**, anything else as **BANK**. The amount does not affect rail selection.

One code, whatever routes your payout

Every destination has a single numeric code (e.g. 1001). We translate it into whatever the provider routing your payout expects, including its exact capitalisation — you never deal with provider-specific code spaces. Legacy raw provider codes you may already be sending (BCA, 0007, Dana, ...) keep working as aliases, but new integrations should use the codes below. Bank names are never accepted in place of a code.

E-wallets

BANK CODE	WALLET
1901	OVO
1902	DANA
1903	GoPay
1904	LinkAja
1905	ShopeePay

Banks — full directory

BANK CODE	BANK NAME
1001	Bank Rakyat Indonesia
1002	Bank Mandiri (Persero) Tbk
1003	Bank Negara Indonesia
1004	Bank Danamon Indonesia Tbk
1005	PT Bank Permata Tbk
1006	PT Bank Permata Tbk Syariah
1007	Bank Central Asia Tbk
1008	PT Bank Internasional Indonesia Tbk
1009	PT Bank Panin Tbk
1010	PT Bank CIMB Niaga Tbk
1011	PT Bank UOB Buana Tbk
1012	PT Bank OCBC NISP Tbk
1013	Citibank N.A.
1014	Bank of America NA
1015	PT Bank Windu Kentjana International Tbk
1016	PT Bank Artha Graha Internasional Tbk
1017	MUFG Bank, Ltd. (Bank of Tokyo-Mitsubishi UFJ)
1018	DBS Bank Ltd.
1019	Bank Resona Perdania
1020	Bank Mizuho Indonesia
1021	Standard Chartered Bank
1022	PT Bank Capital Indonesia Tbk
1023	Bank BNP Paribas Indonesia
1024	PT Bank ANZ Indonesia (ANZ Panin Bank)

1025	Bank of China Limited
1026	PT Bank Bumi Arta Tbk
1027	HSBC Bank (Indonesia)
1028	Bank Antardaerah
1029	PT Bank Rabobank International Indonesia
1030	PT Bank JTrust Indonesia
1031	PT Bank Mayapada Internasional Tbk
1032	Bank Jawa Barat dan Banten Tbk
1033	Bank DKI
1034	Bank Pembangunan Daerah Istimewa Yogyakarta
1035	Bank Jateng
1036	Bank Jatim
1037	Bank Jambi
1038	Bank Jambi Syariah
1039	Bank Aceh
1040	Bank Aceh Syariah
1041	Bank Sumut
1042	Bank Nagari
1043	Bank Riau
1044	Bank Riau Syariah
1045	Bank Sumsel Babel
1046	Bank Sumsel Babel Syariah
1047	Bank Lampung
1048	Bank Kalsel
1049	Bank Kalbar
1050	Bank Pembangunan Daerah Kalimantan Timur

1051	Bank Pembangunan Daerah Kalimantan Tengah
1052	Bank Sulawesi Selatan dan Sulawesi Barat
1053	Bank Sulut
1054	Bank Nusa Tenggara Barat
1055	Bank Nusa Tenggara Barat Syariah
1056	Bank Pembangunan Daerah Bali
1057	Bank Nusa Tenggara Timur
1058	Bank Maluku
1059	Bank Pembangunan Daerah Papua
1060	Bank Bengkulu
1061	Bank Sulawesi Tengah
1062	Bank Sultra
1063	Bank Banten
1064	Bank Nusantara Parahyangan
1065	Bank of India Indonesia
1066	Bank Muamalat Indonesia
1067	Bank Mestika
1068	Shinhan Bank
1069	PT Bank Sinarmas Tbk
1070	PT Bank Maspion Indonesia Tbk
1071	Bank Ganesha
1072	Industrial and Commercial Bank of China (ICBC)
1073	PT Bank QNB Kesawan Tbk
1074	Bank Tabungan Negara
1075	Bank Woori Saudara 1906 Tbk
1076	PT Bank Tabungan Pensiunan Nasional Tbk

1077	PT Bank Victoria Syariah
1078	Bank Jabar Banten Syariah
1079	Bank Mega Tbk
1080	Bank KB Bukopin (Bukopin)
1081	Bank Syariah Indonesia
1082	Bank Jasa Jakarta
1083	KEB Hana Bank
1084	PT Bank MNC Internasional Tbk
1085	Bank Neo Commerce (Yudha Bhakti)
1086	Bank Raya Indonesia (BRI Agroniaga)
1087	Bank SBI Indonesia (Indomonex)
1088	Bank Royal
1089	Bank Nationalnobu
1090	Bank Mega Syariah
1091	Bank Ina Perdana
1092	Bank Panin Dubai Syariah (Panin Syariah)
1093	Bank Prima Master
1094	Bank KB Bukopin Syariah (Syariah Bukopin)
1095	Bank Sahabat Sampoerna
1096	Bank Oke Indonesia (Dinar Indonesia)
1097	Bank Amar Indonesia
1098	Bank Seabank Indonesia
1099	Bank Central Asia Syariah
1100	PT Bank Artos Indonesia
1101	Bank Tabungan Pensiunan Nasional Syariah
1102	Bank Multiarta Sentosa

1103	PT Bank Mayora Indah Tbk
1104	PT Bank Index Selindo
1105	Central Nasional Bank
1106	PT Bank Mantap Sejahtera
1107	PT Bank Victoria International
1108	PT Bank Harda Internasional Tbk
1109	BPR KS (Karyajatnika Sadaya)
1110	PT Bank IFI
1111	Bank Aladin Syariah
1112	China Trust Commercial Bank, Indonesia Branch
1113	Commonwealth Bank of Australia
1114	Bank BPD DIY Syariah
1115	Bank Airway International
1502	BANK ABN AMRO
1504	BANK AKITA
1505	BANK ANDARA
1506	BANK ARTA NIAGA KENCANA
1507	BANK BISNIS INTERNASIONAL
1512	BANK BUMIPUTERA
1513	BANK CCB INDONESIA
1516	BANK CREDIT AGRICOLE INDOSUEZ
1517	BANK DIPO INTERNATIONAL
1519	BANK EKSEKUTIF
1520	BANK EKSPOR INDONESIA
1521	BANK FAMA INTERNASIONAL
1522	BANK FINCONESIA

1525	BANK HARMONI INTERNATIONAL
1527	BANK IBK INDONESIA TBK
1528	BANK IFI
1530	BANK INDONESIA
1531	BANK KEPPEL TATLEE BUANA
1533	BANK KESEJAHTERAAN EKONOMI
1537	BANK MAYBANK INDOCORP
1538	BANK MERINCORP
1539	BANK METRO EXPRESS
1540	BANK MITRANIAGA
1542	BANK MULTICOR TBK.
1556	BANK PEMBANGUNAN DAERAH SUMATERA BARAT
1557	BANK PERSYARIKATAN INDONESIA
1558	BANK PURBA DANARTA
1559	BANK SINAR HARAPAN BALI
1560	BANK SUMITOMO MITSUI INDONESIA
1562	BANK SWAGUNA
1573	BPD SUMBAR
1574	DEUTSCHE BANK AG.
1575	ING INDONESIA BANK
1576	JP. MORGAN CHASE BANK
1581	PT AIRPAY INTERNATIONAL
1594	PT. BANK DANAMON INDONESIA UUS
1595	PT. BANK DKI UUS
1597	PT. BANK JAGO TBK UUS
1599	PT. BANK OCBC NISP UUS

1600	PT. BANK OF CHINA (HONG KONG)
1605	PT. BANK SINARMAS TBK. - UUS
1606	PT. BANK SULSELBAR UUS
1608	PT. BANK TABUNGAN NEGARA UUS
1614	PT. BPD JAWA TENGAH UUS
1615	PT. BPD JAWA TIMUR UUS
1616	PT. BPD KALIMANTAN BARAT UUS
1617	PT. BPD KALIMANTAN SELATAN UUS
1620	PT. BPD KALTIM DAN KALTARA UUS
1626	PT. ESPAY DEBIT INDONESIA KOE
1627	PT. KSEI
1628	PT.BPD SUMATERA UTARA UUS
1629	THE BANGKOK BANK COMP. LTD
2001	American Express Bank
2002	Artajasa Pembayaran Elektronik
2003	Bank Eka (BPR EKA)
2004	Bank Sri Partha
2005	Blu by BCA Digital

If a destination is not available on your account

Some destinations are served by only part of our provider network. A payout to a destination your routing cannot serve is rejected with a clear error — money never moves on a rejected destination. If you plan to rely on a less common bank, confirm it with support before going live.

The live version of this directory is available at `GET /api/merchant/banks` (same authentication as every merchant endpoint). It returns `{ count, banks: [ { code, name, type, universal } ] }` — fetch it at integration time instead of hard-coding this table.

VA Pay-in Banks

Collection banks for the bankCode field when paymentMethodName is VA

Not the same list as payouts

These are collection banks — the banks that can issue a virtual account for you to be paid into. They are a much smaller set than the payout destinations above, and a payout code is not valid here.

VA collection banks

BANK CODE	BANK NAME
BCA	Bank Central Asia
BRI	Bank Rakyat Indonesia
BNI	Bank Negara Indonesia
MANDIRI	Bank Mandiri
CIMB	CIMB Niaga

bankCode is required when paymentMethodName is VA and ignored for every other method. Your account must also have the bank enabled: we reject a VA pay-in for a bank outside your configured list, and reject every VA pay-in if no banks have been enabled yet. Contact support to change which banks you can collect through.